

TCS-504 — SYSTEM DESIGN ASSIGNMENT-1

Movie Ticket Booking System — C++ Console Application

Course: B.Tech. CSE, Semester 5

Subject Code: TCS-504

Submission Date: 07-September-2026

1. Requirement Analysis

The assignment asks for a small, menu-driven movie ticket booking system for a single cinema. The customer must be able to view movies, choose a show, see seat availability, book seats, pay, receive a ticket, and cancel a booking. These are the eight required features in the supplied assignment.

ID	Functional Requirement
FR1	The system shall display all movies currently playing, including their title, language and duration.
FR2	For a selected movie, the system shall display all available shows with screen number and start time.
FR3	For a selected show, the system shall display every seat with its seat number and AVAILABLE/BOOKED status.
FR4	The system shall allow a customer to select one or more seats and shall reject a seat if it is already BOOKED.
FR5	The system shall calculate the booking amount according to seat type: SILVER ₹150, GOLD ₹250 and PLATINUM ₹400.
FR6	The system shall accept UPI, Card or Cash payment. If payment returns failure, the booking shall not be confirmed and temporarily reserved seats shall be released.
FR7	For a successful booking, the system shall print a ticket containing booking ID, movie, screen, time, seat numbers, total amount and payment method.
FR8	The system shall allow cancellation of a confirmed booking and shall make all seats in that booking AVAILABLE again.

Non-Functional Requirements

ID	Requirement
NFR1 — Modularity	Each class is placed in its own .cpp file, as required; responsibilities are separated into entity, payment, pricing, printing and service classes.
NFR2 — Extensibility	Payment is abstract. A future NetBankingPayment can inherit from Payment without changing the BookingService payment algorithm.
NFR3 — Input validation	Invalid menu choices, invalid seat numbers, duplicate seats and already-booked seats are rejected with clear messages without crashing.
NFR4 — Maintainability	Intention-revealing names, small methods, constants through Seat::price(), no duplicated booking logic, and single-responsibility classes are used.

The NFR choices follow the assignment hints concerning modularity, extensibility and validation.

2. Noun–Verb Analysis

Important nouns from the problem are treated as candidate classes; nouns that represent a view/action rather than an independent entity are deliberately rejected. The assignment explicitly gives Movie and Seat as classes and says that 'seat layout' should be a print method rather than a class.



Noun	Class?	Reason	Main methods
Movie	Yes	Has identity and movie data.	getTitle(), getLanguage(), getDurationMinutes()
Seat	Yes	Represents one physical seat with number, type and price.	getNumber(), getType(), getPrice()
Seat layout	No	A view of a Show's seats, not an independent thing.	Show::displaySeats()
Screen	Yes	Represents one auditorium and owns its seats.	add/create seats, findSeat()
Cinema	Yes	Represents the theatre and owns screens.	addScreen(), findScreen()
Show	Yes	Represents one movie screening on one screen at one time.	findShowSeat(), displaySeats()
ShowSeat	Yes	Stores status of one physical seat for one show.	bookSeat(), cancelSeat(), isAvailable()

Customer	Yes	Stores customer identity information.	getName(), getPhone()
Booking	Yes	Stores one booking's identity, seats, amount and status.	confirm(), cancel()
Payment	Yes	Payment contract shared by payment methods.	pay(), methodName()
UPI/Card/Cash	Yes	Concrete payment behaviours.	pay()
PriceCalculator	Yes	Has one focused pricing responsibility.	calculate()
TicketPrinter	Yes	Has one focused printing responsibility.	print()
BookingService	Yes	Orchestrates booking, payment and cancellation.	bookTicket(), cancelBooking()
Cinema/main menu	Yes	Handles console menu and input.	run(), input helpers

3. Class Responsibilities

Class	Knows	Does	Must NOT do
Movie	Knows title, language, duration.	Provides movie data.	Must NOT manage shows, seats, payments or tickets.
Seat	Knows physical seat number and type.	Provides seat identity and price by type.	Must NOT know whether it is booked for a particular show.
Screen	Knows screen number and its physical seats.	Creates/owns its seats.	Must NOT manage bookings or payments.
Cinema	Knows cinema name and screens.	Creates/owns screens.	Must NOT calculate ticket prices or process payments.
Show	Knows movie, screen, start time and show seats.	Represents one screening and displays its seats.	Must NOT process payment or print tickets.
ShowSeat	Knows one physical seat and its status for one show.	Books/releases its own status.	Must NOT calculate a whole booking total.
Customer	Knows name and phone.	Provides customer information.	Must NOT own the booking service.

Booking	Knows booking ID, customer, show, seats, amount and status.	Changes booking state to confirmed/cancelled.	Must NOT print its own ticket or process payment.
Payment	Knows only the payment contract.	Defines pay(amount).	Must NOT contain cinema/show/seat logic.
UpiPayment/CardPayment/CashPayment	Knows the data needed for its own payment simulation.	Implements Payment::pay().	Must NOT manage bookings or seats.
PriceCalculator	Knows pricing rules.	Calculates total for selected seats.	Must NOT book seats or process payment.
TicketPrinter	Knows ticket formatting rules.	Prints a ticket.	Must NOT change booking/seat/payment state.
BookingService	Knows the booking workflow and stored	Validates, reserves, pays, confirms/releases and cancels.	Must NOT implement individual payment algorithms.
	bookings.		
CinemaMenu	Knows console interaction.	Displays menu and reads validated input.	Must NOT contain pricing/payment algorithms.

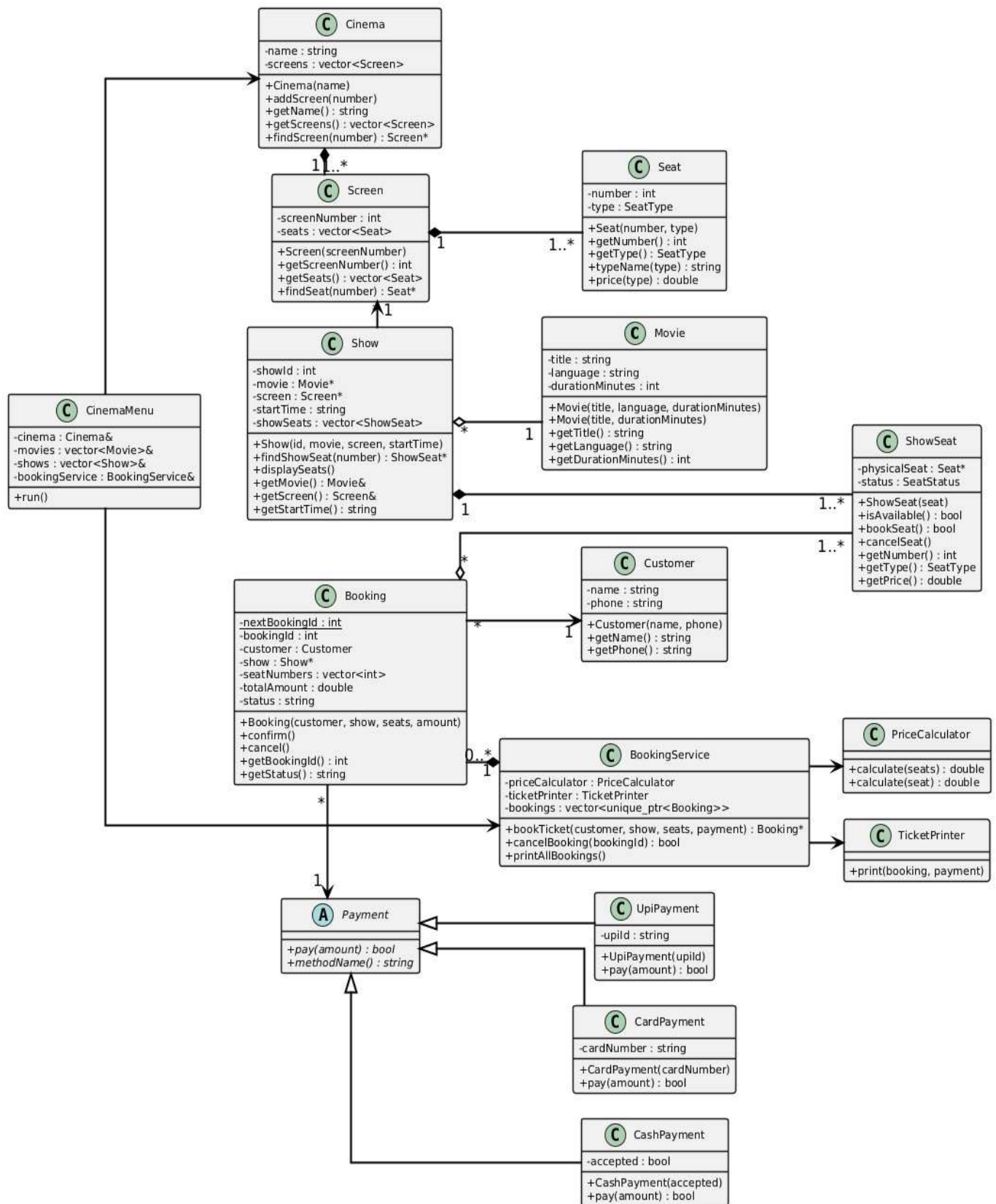
4. Relationship Table

Pair	Choice	Lifetime justification
Cinema — Screen	Composition (◆)	Cinema creates and owns its Screen objects. If the Cinema is destroyed in this model, its screens are destroyed with it.
Screen — Seat	Composition (◆)	Screen creates and owns its physical Seat objects. A seat belongs to that screen in this model, so it dies with the screen.
Show — Movie	Aggregation (◇)	A Show only refers to an existing Movie; destroying a Show must not destroy the Movie.
Show — Screen	Association (—▶)	A Show refers to an existing Screen as its auditorium; the Screen is an independently existing cinema resource.

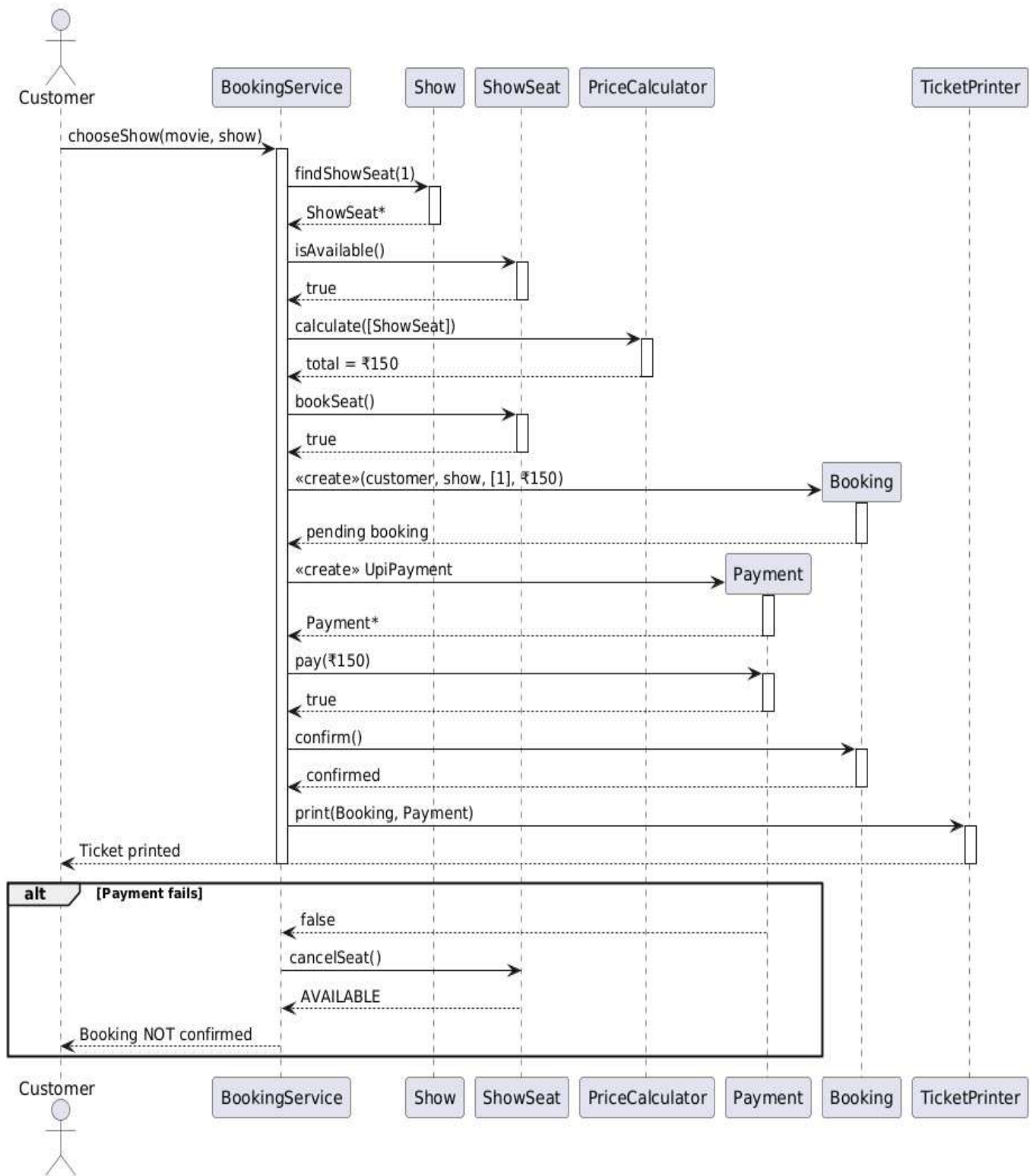
Show — ShowSeat	Composition (◆)	Each Show creates and owns its ShowSeat status objects. Destroying the Show removes those per-show status objects.
Booking — Customer	Association (—▶)	A booking records customer information/use of a customer; neither object's lifetime depends on the other.
Booking — ShowSeat	Aggregation (◇)	A Booking refers to existing ShowSeat objects; destroying a Booking must not destroy the seats/status objects belonging to the Show.
Booking — Payment	Association (—▶)	Payment is supplied to the booking operation and is not owned as a long-lived part of Booking.
Payment — UpiPayment	Inheritance (—▷)	UpiPayment is a specialized Payment implementation.
BookingService — Booking	Composition (◆)	This implementation stores confirmed Booking objects inside the service. Destroying the service destroys its owned booking records.

The assignment requires every relationship choice to be justified using the lifetime test and gives Composition/Aggregation examples such as Cinema→Screen, Screen→Seat, Show→ShowSeat and Show→Movie.

5. Class Diagram



6. Sequence Diagram — Book 1 Seat and Pay by UPI



7. Modular Working Code

The complete C++ project is supplied separately in this folder/ZIP. It follows the course rule of one class per file and no header files. main.cpp includes the .cpp class files in dependency order so the complete program can be compiled as one translation unit. The assignment explicitly requires one class per file and no headers.

Files:

- Booking.cpp
- BookingService.cpp
- CardPayment.cpp
- CashPayment.cpp
- Cinema.cpp
- CinemaMenu.cpp
- Customer.cpp
- Movie.cpp
- Payment.cpp
- PriceCalculator.cpp
- Screen.cpp
- Seat.cpp
- Show.cpp
- ShowSeat.cpp
- TicketPrinter.cpp
- UpiPayment.cpp
- main.cpp Compile with C++17: `g++ -std=c++17 main.cpp -o MovieBooking MovieBooking`

8. Concepts Demonstrated

Concept	Where it appears
Encapsulation	Private data in Movie, Seat, ShowSeat, Booking, Customer and payment classes; state changes happen through methods such as bookSeat() and cancelSeat().
Abstraction	Payment is an abstract class with pure virtual pay(double amount).
Inheritance	UpiPayment, CardPayment and CashPayment inherit from Payment.
Runtime polymorphism	BookingService uses Payment* and calls pay(total), selecting the concrete implementation at runtime.
Compile-time polymorphism	Movie has overloaded constructors and PriceCalculator has overloaded calculate() methods.
Static member	Booking::nextBookingId is static and generates unique booking IDs.
this keyword	Constructors such as Movie and Customer use this-
	style member initialization; the implementation also uses constructor initializer lists to clearly bind current-object data.

Composition	Cinema→Screen, Screen→Seat and Show→ShowSeat are modeled with owned objects.
Aggregation	Show references existing Movie/Screen resources; Booking references existing ShowSeat status objects.

These concepts match the assignment's required OOP list and implementation hints.

9. SOLID Mapping

Principle	Application
S — Single Responsibility	Movie stores movie data; PriceCalculator calculates prices; TicketPrinter prints; CinemaMenu handles console input; BookingService orchestrates the workflow.
O — Open/Closed	Adding NetBankingPayment can be done by creating another Payment subclass. BookingService depends on Payment rather than a concrete payment class.
L — Liskov Substitution	Every payment subclass implements Payment::pay() and can be used through Payment* without special setup.
D — Dependency Inversion	BookingService receives a Payment& abstraction instead of constructing UpiPayment/CardPayment/CashPayment itself.

One deliberately NOT forced: Interface Segregation is not artificially added because the small Payment interface only needs pay() and methodName() for this scope. Adding unrelated methods such as refund() would make the base interface larger without being required. This directly addresses the assignment's request to state one thing deliberately not done.

10. Edge Cases and Demo Tests

Test	Expected result
Already booked seat	Book seat 1 successfully, then attempt seat 1 again for the same show. Expected: 'already BOOKED' and no state change.
Failed payment	During payment, enter FAIL as the UPI ID or card number, or choose Cash and answer No. Expected: payment failure, booking not confirmed and all temporarily reserved seats become AVAILABLE.
Cancellation	Cancel a confirmed booking by its booking ID. Expected: booking becomes CANCELLED and its seats become AVAILABLE.
Invalid seat/menu	Enter a value outside the allowed range. Expected: clear validation message and program continues.

Duplicate seat	Enter the same seat number twice in one booking. Expected: booking rejected before any seat is changed.
----------------	--

These tests cover the four mandatory edge cases stated in the assignment plus duplicate-seat validation.

11. Clean-Code Checklist

- Intention-revealing names such as `bookedSeatCount`-style descriptive naming; no `list1/temp1` naming.
- Small focused functions; no large monolithic booking method.
- At most two practical indentation levels in the main workflow.
- Constants are used for seat prices through `Seat::price()` instead of scattering magic numbers.
- No repeated payment/seat-release workflow.
- Methods have clear side effects: `displaySeats()` only displays; `bookSeat()/cancelSeat()` change status.
- Responsibilities are separated according to the assignment's one-class/one-responsibility guidance.

These choices follow the supplied clean-code checklist.

12. Demo Run — Successful Booking and Cancellation

```
===== PVR Dehradun MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit

```
Enter choice: 1
```

```
Currently Playing
```

```
Interstellar | English | 169 min
```

```
3 Idiots | Hindi | 170 min
```

```
Demon Slayer | Japanese | 120 min
```

```
===== PVR Dehradun MOVIE TICKET BOOKING =====
```

1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit

```
Enter choice: 2
```

```
Currently Playing
```

```
Interstellar | English | 169 min
```

```
3 Idiots | Hindi | 170 min
```

```
Demon Slayer | Japanese | 120 min
```

```
Choose movie (1-3): 1
```

```
Shows for Interstellar
```

```
1. Screen 1 | 10:00 AM
```

```
2. Screen 2 | 06:30 PM
```

```
Choose show: 1
```

===== PVR Dehradun MOVIE TICKET BOOKING =====

1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit

Enter choice: 3

Currently Playing

Interstellar | English | 169 min

3 Idiots | Hindi | 170 min

Demon Slayer | Japanese | 120 min

Choose movie (1-3): 1

Shows for Interstellar

1. Screen 1 | 10:00 AM

2. Screen 2 | 06:30 PM

Choose show: 1

Seat Layout (S=Silver, G=Gold, P=Platinum)

[1 AVAILABLE]	[2 AVAILABLE]	[3 AVAILABLE]	[4 AVAILABLE]
[5 AVAILABLE]	[6 AVAILABLE]	[7 AVAILABLE]	[8 AVAILABLE]
[9 AVAILABLE]	[10 AVAILABLE]	[11 AVAILABLE]	[12 AVAILABLE]
[13 AVAILABLE]	[14 AVAILABLE]	[15 AVAILABLE]	[16 AVAILABLE]
[17 AVAILABLE]	[18 AVAILABLE]	[19 AVAILABLE]	[20 AVAILABLE]

Prices: SILVER=Rs.150, GOLD=Rs.250, PLATINUM=Rs.400

===== PVR Dehradun MOVIE TICKET BOOKING =====

1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit

Enter choice: 4

Currently Playing

Interstellar | English | 169 min

3 Idiots | Hindi | 170 min

Demon Slayer | Japanese | 120 min

Choose movie (1-3): 1

Shows for Interstellar

1. Screen 1 | 10:00 AM

2. Screen 2 | 06:30 PM

Choose show: 1

Seat Layout (S=Silver, G=Gold, P=Platinum)

[1 AVAILABLE] [2 AVAILABLE] [3 AVAILABLE] [4 AVAILABLE]

[5 AVAILABLE] [6 AVAILABLE] [7 AVAILABLE] [8 AVAILABLE]

[9 AVAILABLE] [10 AVAILABLE] [11 AVAILABLE] [12 AVAILABLE]

[13 AVAILABLE] [14 AVAILABLE] [15 AVAILABLE] [16 AVAILABLE]

[17 AVAILABLE] [18 AVAILABLE] [19 AVAILABLE] [20 AVAILABLE]

Prices: SILVER=Rs.150, GOLD=Rs.250, PLATINUM=Rs.400

Customer name: Anchal

Phone number: 8539364268

How many seats? 4

Enter seat number 1 (1-20): 5

Enter seat number 2 (1-20): 6

Enter seat number 3 (1-20): 7

Enter seat number 4 (1-20): 8

Payment Method

1. UPI
2. Card
3. Cash

Choose payment method: 1

Enter UPI ID (type FAIL to simulate failure): anchal@oksbi

Total amount: Rs.600.00

Processing UPI payment of Rs.600.00...

UPI payment successful.

=====

TICKET

=====

Booking ID : 1001

Customer : Anchal

Movie : Interstellar

Screen : 1

Start Time : 10:00 AM

Seats : 5, 6, 7, 8

Amount : Rs.600.00

Payment : UPI

Status : CONFIRMED

=====

===== PVR Dehradun MOVIE TICKET BOOKING =====

1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit

Enter choice: 5

Enter booking ID to cancel (1001+): 1001

Booking 1001 cancelled. Seats are AVAILABLE again.

```
===== PVR Dehradun MOVIE TICKET BOOKING =====
1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit
Enter choice: 6

Booking Records
ID: 1001 | Movie: Interstellar | Seats: 5 6 7 8 | Amount: Rs.600.00 | Status: CANCELLED

===== PVR Dehradun MOVIE TICKET BOOKING =====
1. List movies
2. List shows
3. Display seat layout
4. Book ticket
5. Cancel booking
6. Show booking records
7. Exit
Enter choice: |
```

13. Conclusion

The submitted program implements the exact single-cinema movie-ticket scope using object-oriented C++. It separates data, behaviour and orchestration into the required classes, demonstrates the requested OOP concepts and SOLID principles, validates invalid input, handles failed payments safely, prints tickets and releases seats after cancellation.